

SÃO PAULO TECH SCHOOL - SPTECH

ARTHUR BISPO DOS SANTOS

ACERVOTECH

Documentação técnica da API REST e da integração com React

São Paulo
2026

ARTHUR BISPO DOS SANTOS

ACERVOTECH

Documentação técnica da API REST e da integração com React

Documentação técnica apresentada à São Paulo Tech School como parte do Projeto Individual Integrador das disciplinas de Front-end e Programação Web.

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO	3
1.1 Objetivo e escopo	3
1.2 Organização do desenvolvimento	3
1.3 Estrutura da documentação	3
2 ORGANIZAÇÃO E INTEGRAÇÃO	4
2.1 Estrutura do repositório	4
2.2 Comunicação entre as aplicações	4
3 EXECUÇÃO LOCAL	5
3.1 Pré-requisitos	5
3.2 Inicialização do back-end	5
3.3 Inicialização do front-end	5
4 MODELO DE DADOS E PERSISTÊNCIA	6
4.1 Criação da tabela	6
4.2 Configuração e duração dos registros	6
5 CONTRATO DOS ENDPOINTS	7
5.1 Listagem de livros	7
5.2 Significado dos códigos	7
6 CONSULTA POR CATEGORIA	8
6.1 Requisição e resposta	8
6.2 Ausência de resultados e interface	8
7 CADASTRO DE LIVROS	9
7.1 Corpo da requisição	9
7.2 Resposta de sucesso	9
8 VALIDAÇÕES E TRATAMENTO DE ERROS	10
8.1 Requisição inválida	10
8.2 Carregamento no cliente	10
9 ROTEIRO DE VERIFICAÇÃO	11
9.1 Critérios de aceitação	11
10 SOLUÇÃO DE PROBLEMAS E ENTREGA	12
10.1 Organização da documentação	12
10.2 Limitações da aplicação	12
REFERÊNCIAS	13

1 INTRODUÇÃO

Desenvolvi o AcervoTech como projeto individual de integração entre as disciplinas de Front-end e Programação Web. A aplicação permite cadastrar e consultar livros por meio de um cliente React integrado a uma API REST em Java com Spring Boot.

Nesta documentação, apresento a organização da solução, a execução local, a estrutura do banco de dados, o contrato dos endpoints e as regras de validação. Reuni também um roteiro de verificação manual. O README concentra as instruções essenciais para iniciar a aplicação.

1.1 Objetivo e escopo

A aplicação contempla o cadastro de livros com cinco campos, a listagem do acervo e a busca por categoria. Não fazem parte do fluxo documentado as operações de edição, exclusão, empréstimo, autenticação ou consulta individual por identificador.

O armazenamento é realizado em uma tabela relacional por meio de JdbcTemplate. A lista exibida no cliente é obtida da API; as opções fixas de categoria do formulário não substituem os registros persistidos no banco.

1.2 Organização do desenvolvimento

Organizei o projeto em duas pastas: cliente, responsável pela interface, e api, responsável pelo processamento das requisições e pelo acesso ao banco. Defini o livro como recurso principal e estabeleci um contrato de integração com operações de cadastro, listagem e filtro por categoria.

Defini as validações de preenchimento e tamanho de acordo com os campos da tabela. Para orientar a verificação do fluxo, apresento cenários de cadastro e consulta com seus resultados esperados.

1.3 Estrutura da documentação

Organizei o documento em seções sobre arquitetura, execução, persistência, endpoints, validações e verificação. Adotei as orientações aplicáveis da ABNT NBR 14724:2024 para a apresentação acadêmica, com margens, espaçamento e paginação padronizados (Associação Brasileira de Normas Técnicas, 2024).

2 ORGANIZAÇÃO E INTEGRAÇÃO

Quadro 1 - Componentes da solução

Componente	Responsabilidade
React e JSX	Construção das telas de início, cadastro e consulta.
CSS Modules	Estilização dos componentes e páginas.
Axios	Envio de requisições HTTP à API.
React Router DOM	Navegação entre as páginas do cliente.
Spring Boot e Java 21	Inicialização da API e processamento das requisições.
JdbcTemplate	Execução de consultas e inserções SQL.
H2	Armazenamento relacional dos livros durante a execução.

Fonte: elaboração própria (2026).

2.1 Estrutura do repositório

Na organização do back-end, a pasta **api** reúne o pom.xml, o código Java e os recursos de configuração. Em src/main/resources estão application.properties e schema.sql. O modelo Livro representa os dados, enquanto LivroController recebe as requisições, valida o cadastro e executa o acesso ao banco.

A pasta **cliente** contém o projeto React. As páginas ficam em src/pages, os componentes compartilhados em src/components e a configuração de comunicação em src/services/api.js. A página CadastroLivro envia o POST; VerificarLivro consulta os registros por GET.

2.2 Comunicação entre as aplicações

Configurei o front-end para utilizar a URL base http://localhost:8080. Ao cadastrar, converte anoPublicacao e quantidade para números, envia o JSON e apresenta o resultado. A API valida os campos, executa o INSERT e retorna o livro com o ID gerado pelo banco. Na consulta, o cliente renderiza a lista recebida.

```
export const api = axios.create({
  baseURL: "http://localhost:8080"
})
```

O controller utiliza @CrossOrigin para permitir acesso entre origens distintas. Se houver restrição de origem, configure o endereço do cliente. Ao alterar a porta da API, atualize também a baseURL do Axios.

3 EXECUÇÃO LOCAL

3.1 Pré-requisitos

É necessário ter JDK 21, Maven, Node.js compatível com as dependências do cliente e npm instalados e disponíveis no terminal. O JDK deve incluir o compilador javac. Os comandos abaixo permitem conferir as instalações; mvn -version também informa qual Java o Maven está utilizando.

```
java -version
javac -version
mvn -version
node -v
npm -v
```

3.2 Inicialização do back-end

Abra a pasta raiz AcervoTech no VS Code. No menu, selecione Terminal > Novo Terminal e confirme que o terminal está na pasta que contém api e cliente. Entre em api e execute o comando Maven:

```
cd api
mvn spring-boot:run
```

Aguarde a inicialização de AcervoTechApplication. Na primeira execução, o Maven poderá baixar as dependências. Mantenha esse terminal aberto. A API estará disponível em <http://localhost:8080>; a consulta pode ser acessada em <http://localhost:8080/livros>.

3.3 Inicialização do front-end

Abra um segundo terminal, mantendo o primeiro em execução. Partindo novamente da raiz AcervoTech, entre em cliente. Na primeira utilização, instale as dependências antes de iniciar o servidor:

```
cd cliente
npm install
npm run dev
```

Nas execuções seguintes, utilize npm run dev dentro de cliente. Se o novo terminal abrir em api, execute cd ../cliente. Abra no navegador o endereço informado pelo Vite, normalmente <http://localhost:5173>. Os dois terminais devem permanecer ativos. Para encerrar, pressione Ctrl + C em cada um.

4 MODELO DE DADOS E PERSISTÊNCIA

Defini o livro como recurso principal da aplicação. Seus cinco campos de cadastro possuem correspondência na tabela livro. O identificador é gerado pelo banco e permite distinguir os registros retornados à interface.

Quadro 2 - Correspondência entre JSON e banco

Campo JSON	Coluna SQL	Tipo SQL
id	id	INT; chave primária gerada
titulo	titulo	VARCHAR(150) NOT NULL
autor	autor	VARCHAR(100) NOT NULL
categoria	categoria	VARCHAR(50) NOT NULL
anoPublicacao	ano_publicacao	INT NOT NULL
quantidade	quantidade	INT NOT NULL

Fonte: elaboração própria (2026).

4.1 Criação da tabela

O arquivo `api/src/main/resources/schema.sql` contém a criação da tabela. Na configuração atual do banco em memória, o script é utilizado na inicialização.

```
CREATE TABLE livro (
  id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  titulo VARCHAR(150) NOT NULL,
  autor VARCHAR(100) NOT NULL,
  categoria VARCHAR(50) NOT NULL,
  ano_publicacao INT NOT NULL,
  quantidade INT NOT NULL
);
```

4.2 Configuração e duração dos registros

O `application.properties` contém `spring.application.name=AcervoTech`. Com H2 disponível e sem URL de conexão definida, a configuração automática do Spring Boot utiliza um banco embarcado em memória. Não é necessário instalar um servidor de banco separado (Spring, [s. d.]).

Os livros permanecem disponíveis enquanto a API está ligada. Reiniciá-la elimina os dados do banco em memória; atualizar somente o navegador não os elimina. Conservar registros entre execuções exigiria armazenamento em arquivo ou outro banco, ainda não configurado.

5 CONTRATO DOS ENDPOINTS

A URL base da API é `http://localhost:8080`. Os cadastros utilizam o cabeçalho `Content-Type: application/json`. As respostas de sucesso com conteúdo são representadas em JSON. Os números dos identificadores nos exemplos são ilustrativos.

Quadro 3 - Operações disponíveis

Método	Caminho	Sucesso
GET	/livros	200 OK
GET	/livros/categoria/{categoria}	200 OK
POST	/livros	201 Created

Fonte: elaboração própria (2026).

5.1 Listagem de livros

O endpoint `GET /livros` retorna todos os livros cadastrados. Não exige corpo de requisição nem parâmetros. Uma consulta sem registros retorna 200 OK e uma lista vazia, representada por `[]`.

```
curl -i http://localhost:8080/livros
```

Exemplo de resposta com status 200 OK:

```
[
  {
    "id": 1,
    "titulo": "O Hobbit",
    "autor": "J. R. R. Tolkien",
    "categoria": "Fantasia",
    "anoPublicacao": 1937,
    "quantidade": 3
  }
]
```

A ordem dos registros não é definida pelo contrato, pois a consulta SQL atual não utiliza `ORDER BY`. O cliente deve exibir os dados recebidos sem depender de uma ordenação específica.

5.2 Significado dos códigos

No contrato da API, defini 200 para consultas, 201 para criação e 400 para falhas de validação. Para os GETs, o retorno correspondente é `ResponseEntity.ok(livros)`. A consulta de uma categoria sem resultados retorna uma lista vazia com 200. O fluxo atual não requer uma resposta 204.

6 CONSULTA POR CATEGORIA

O endpoint `GET /livros/categoria/{categoria}` permite consultar uma parte do acervo. A categoria é um parâmetro obrigatório no caminho da URL, do tipo String. A operação não recebe corpo de requisição.

Quadro 4 - Parâmetro da consulta por categoria

Parâmetro	Local	Exemplo
categoria	Caminho da URL	Fantasia

Fonte: elaboração própria (2026).

6.1 Requisição e resposta

Utilize a grafia cadastrada, como Fantasia. A consulta utiliza igualdade no SQL e não implementa busca parcial por texto.

```
curl -i http://localhost:8080/livros/categoria/Fantasia
```

Exemplo de resposta com status 200 OK:

```
[
  {
    "id": 1,
    "titulo": "O Hobbit",
    "autor": "J. R. R. Tolkien",
    "categoria": "Fantasia",
    "anoPublicacao": 1937,
    "quantidade": 3
  }
]
```

6.2 Ausência de resultados e interface

Quando não houver correspondência, o corpo será []. O cliente deve apresentar a mensagem de ausência de livros, sem tratar a lista vazia como falha de comunicação.

Na tela VerificarLivro, o usuário seleciona a categoria e aciona Buscar. Com a opção Todas as categorias selecionada, o cliente utiliza `GET /livros`. Com uma categoria definida, utiliza o endpoint de filtro.

As opções do select são Romance, Fantasia, Terror, Suspense, Biografia e Tecnologia. A validação de texto da API não restringe o cadastro a essa lista; um valor diferente que respeite as regras de preenchimento e tamanho poderá ser recebido diretamente pela API.

7 CADASTRO DE LIVROS

O endpoint POST /livros recebe um objeto com os cinco campos obrigatórios. O ID não precisa ser enviado: ele é gerado pelo banco e inserido no objeto devolvido após o cadastro.

7.1 Corpo da requisição

```
{
  "titulo": "O Hobbit",
  "autor": "J. R. R. Tolkien",
  "categoria": "Fantasia",
  "anoPublicacao": 1937,
  "quantidade": 3
}
```

No Postman, selecione POST, informe `http://localhost:8080/livros` e utilize Body > raw > JSON. Cole o objeto acima e envie. Para executar por curl no Linux/bash:

```
curl -i -X POST http://localhost:8080/livros \
-H "Content-Type: application/json" \
-d '{
  "titulo": "O Hobbit",
  "autor": "J. R. R. Tolkien",
  "categoria": "Fantasia",
  "anoPublicacao": 1937,
  "quantidade": 3
}'
```

7.2 Resposta de sucesso

O contrato define o retorno 201 Created, com o livro cadastrado:

```
{
  "id": 1,
  "titulo": "O Hobbit",
  "autor": "J. R. R. Tolkien",
  "categoria": "Fantasia",
  "anoPublicacao": 1937,
  "quantidade": 3
}
```

Após o sucesso, a interface apresenta uma confirmação e limpa os campos. Não defini uma regra de unicidade de título na tabela; enviar novamente um cadastro válido pode criar outro registro.

8 VALIDAÇÕES E TRATAMENTO DE ERROS

As regras são aplicadas no servidor antes de persistir o registro. Dessa forma, o uso de Postman ou curl não permite contornar a validação implementada no controller.

Quadro 5 - Regras do cadastro

Campo	Validação
titulo	Obrigatório; não nulo, vazio ou somente espaços; até 150 caracteres.
autor	Obrigatório; não nulo, vazio ou somente espaços; até 100 caracteres.
categoria	Obrigatório; não nulo, vazio ou somente espaços; até 50 caracteres.
anoPublicacao	Obrigatório; inteiro maior que zero.
quantidade	Obrigatório; inteiro maior que zero.

Fonte: elaboração própria (2026).

8.1 Requisição inválida

O JSON a seguir viola a regra de título obrigatório. O resultado esperado é 400 Bad Request, sem corpo, conforme o retorno `ResponseEntity.badRequest().build()` do controller. Nenhum registro deve ser inserido.

```
{
  "titulo": "",
  "autor": "J. R. R. Tolkien",
  "categoria": "Fantasia",
  "anoPublicacao": 1937,
  "quantidade": 3
}
```

Erros de leitura do JSON podem ser tratados pelo próprio Spring. A aplicação não define um formato personalizado de resposta para todas as falhas; por isso, o cliente não deve depender de uma propriedade de mensagem no corpo do erro.

8.2 Carregamento no cliente

No cadastro, utilizo o estado carregando, inicializado com `useState(false)`, ativado antes do POST e desativado em `finally`. Durante a requisição, o botão fica desabilitado e apresenta Cadastrando.... No sucesso, a página exibe a confirmação; na falha, informa que não foi possível cadastrar.

Defini limites de 150 e 100 caracteres para título e autor, respectivamente, na interface e na API.

9 ROTEIRO DE VERIFICAÇÃO

Defini os cenários abaixo para verificar o cadastro, a consulta e o tratamento de erros. O quadro apresenta os resultados esperados para cada entrada. A execução do roteiro consiste em comparar o status recebido com o contrato e consultar o acervo para conferir o efeito de cada requisição.

Quadro 6 - Casos de verificação manual

Cenário	Resultado esperado
Cadastrar um livro válido	201; retorno com ID; registro aparece na consulta.
Consultar o acervo	200; array de livros com os cinco dados cadastrados.
Filtrar uma categoria cadastrada	200; apenas registros correspondentes.
Consultar categoria sem livros	200; corpo []; mensagem de lista vazia no cliente.
Enviar título vazio ou autor só com espaços	400; nenhum registro inserido.
Enviar texto nulo ou omitir campo obrigatório	400; nenhum registro inserido.
Enviar título com 151, autor com 101 ou categoria com 51 caracteres	400; nenhum registro inserido.
Enviar texto no limite, com os demais campos válidos	Cadastro aceito para 150, 100 e 50 caracteres, respectivamente.
Enviar ano ou quantidade igual a zero ou negativo	400; nenhum registro inserido.
Interromper a API e consultar ou cadastrar	Cliente informa falha e encerra o estado de carregamento.
Reiniciar a API com H2 em memória	Registros anteriores deixam de existir.

Fonte: elaboração própria (2026).

9.1 Critérios de aceitação

Como critérios de aceitação, estabeleci o cadastro com retorno 201, as consultas com retorno 200 e a rejeição dos dados inválidos com 400. A interface deve apresentar os registros da API e informar carregamento, sucesso ou falha conforme a operação. O README descreve a execução em dois terminais, sem scripts auxiliares.

10 SOLUÇÃO DE PROBLEMAS E ENTREGA

Quadro 7 - Diagnóstico de execução

Sintoma	Ação
No plugin found for prefix	Conferir a escrita de mvn spring-boot:run e executar na pasta api.
Pacotes do Spring não encontrados	Iniciar a aplicação pelo Maven, carregando as dependências do pom.xml.
Endereço já em uso	Parar a execução anterior da API e manter uma única instância na porta 8080.
Maven ou javac não encontrado	Conferir a instalação do Maven e do JDK 21 no terminal.
Vite não encontrado	Executar npm install em cliente antes de npm run dev.
Falha de comunicação no navegador	Verificar se a API está ligada, a baseURL do Axios e o CORS.
carregando is not defined	Declarar const [carregando, setCarregando] = useState(false) dentro do componente.

Fonte: elaboração própria (2026).

10.1 Organização da documentação

Dividi a documentação entre o README, com apresentação e instruções de execução, e este PDF, com o contrato detalhado da API. O caminho definido para o documento é docs/documentacao-api.pdf. O link relativo utilizado no README é:

[Documentação técnica](docs/documentacao-api.pdf)

A organização da entrega reúne os códigos-fonte nas pastas api e cliente, o script schema.sql, as configurações necessárias e a documentação. O link do repositório GitHub é o meio de envio definido para a atividade.

10.2 Limitações da aplicação

Nesta versão, concentrei o escopo no cadastro e na consulta do acervo. Utilizei H2 em memória, sem conservação dos registros após encerrar a API. A categoria é validada como texto, sem lista fechada no servidor, e as respostas de validação não possuem corpo personalizado.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724:2024: informação e documentação: trabalhos acadêmicos: apresentação**. Versão corrigida de 1 abr. 2025. Rio de Janeiro: ABNT, 2024. Disponível em: https://engenhariaedesenvolvimentosustentavel.ufes.br/sites/engenhariaedesenvolvimentosustentavel.ufes.br/files/field/anexo/abnt_nbr_14724.pdf. Acesso em: 7 set. 2026.

SANTOS, Arthur Bispo dos. **AcervoTech**. Código-fonte e documentação do projeto individual. 2026. Projeto individual desenvolvido nas disciplinas de Front-end e Programação Web.

SÃO PAULO TECH SCHOOL. **Projeto Integrador entre Front-end e Programação Web: projeto individual**. 2026. Enunciados das disciplinas de Front-end e Programação Web.

SPRING. **SQL databases**. Spring Boot Reference Documentation. [S. l.], [s. d.]. Disponível em: <https://docs.spring.io/spring-boot/reference/data/sql.html>. Acesso em: 7 set. 2026.